2.11 C语言中的数据类型及其转换

考纲定位：计算机组成原理 · 第 2 章 数据的表示和运算（2.11） | 重要度 ★★★★★ | 常见考法：选择题/大题（整型转换、符号扩展、截断、int/float/double 转换、精度丢失）

一句话核心：同字长转换按位重新解释，短转长看符号决定零扩展/符号扩展，长转短截断；浮点与整数转换看精度和范围

一、数据类型

1.1 类型

■ 计算机中的数据以二进制的形式存储在寄存器或存储器中。

■ 汇编语言中的数据类型取决于指令操作码。

- □ 存储在寄存器、存储器中的操作数本身没有数据类型，对该数进行何种数据类型的操作完全取决于指令。
- □ 同一个操作数，既可以当作有符号数，也可以当作无符号数；既可以是定点数，也可以是浮点数。

■ 高级语言具有数据类型，下面以C语言为例进行介绍。



注：寄存器中的操作数本身不存其是何种操作类型，对该数进行何种操作，取决于指令

C 语言常见类型

- `char` / `unsigned char`：8 位
- `short` / `unsigned short`：通常 16 位
- `int` / `unsigned int`：通常 32 位
- `float`：32 位 IEEE 754
- `double`：64 位 IEEE 754

易错

- C 标准只规定最小范围，具体位数由实现决定；408 按常见 32 位模型
- `char` 是否有符号由实现决定；408 通常按有符号 `char`
- 寄存器不保存“类型”，类型只决定指令如何解释位串

1.2 整型的取值范围

C语言中整型变量的取值范围

		char（8位）	short（16位）	int（32位）	long（64位）
无符号(unsigned)最大值	机器码	0xFF	0xFFFF	0xFFFF FFFF	0xFFFF FFFF FFFF FFFF
	真值	255	65,535	4,294,967,295	18,446,744,073,709,551,615
有符号最大值	机器码	0x7F	0x7FFF	0x7FFF FFFF	0x7FFF FFFF FFFF FFFF
	真值	127	32,767	2,147,483,647	9,223,372,036,854,775,807
有符号最小值	机器码	0x80	0x8000	0x8000 0000	0x8000 0000 0000 0000
	真值	−128	−32,768	−2,147,483,648	−9,223,372,036,854,775,808
−1	机器码	0xFF	0xFFFF	0xFFFF FFFF	0xFFFF FFFF FFFF FFFF
0	机器码	0x00	0x0000	0x0000 0000	0x0000 0000 0000 0000

■ 无论是无符号数还是有符号数，C语言程序并不检测数据在加、减、乘等运算中产生的溢出现象。

□ 程序员应尽量避免出现这种情况，所编制的应用程序应具有对溢出进行判断的功能。

类型	位数	范围
`char`	8	$-128 \sim 127$
`unsigned char`	8	$0 \sim 255$
`short`	16	$-32768 \sim 32767$
`unsigned short`	16	$0 \sim 65535$
`int`	32	$-2^{31} \sim 2^{31}-1$
`unsigned int`	32	$0 \sim 2^{32}-1$

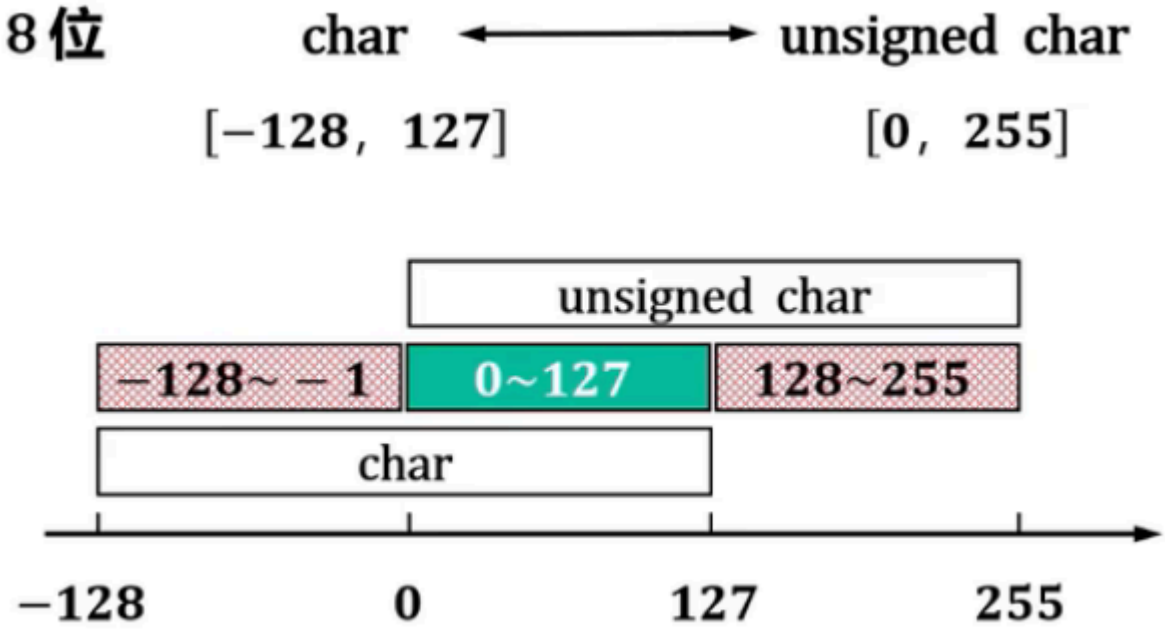
注：C 语言不自动检测溢出，参考 2.11A C语言不检测数据溢出的特性

二、类型转换

2.1 整型之间的转换

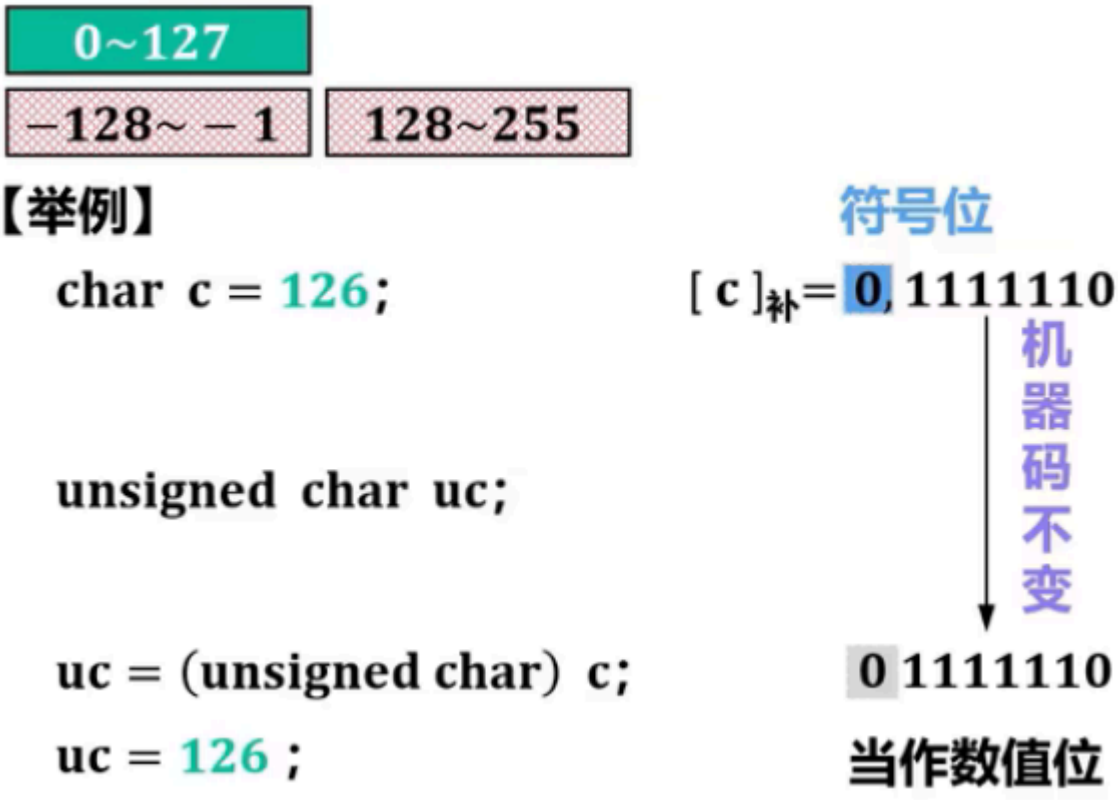
2.1.1 相同字长之间的转换

以 `char` 与 `unsigned char` 举例

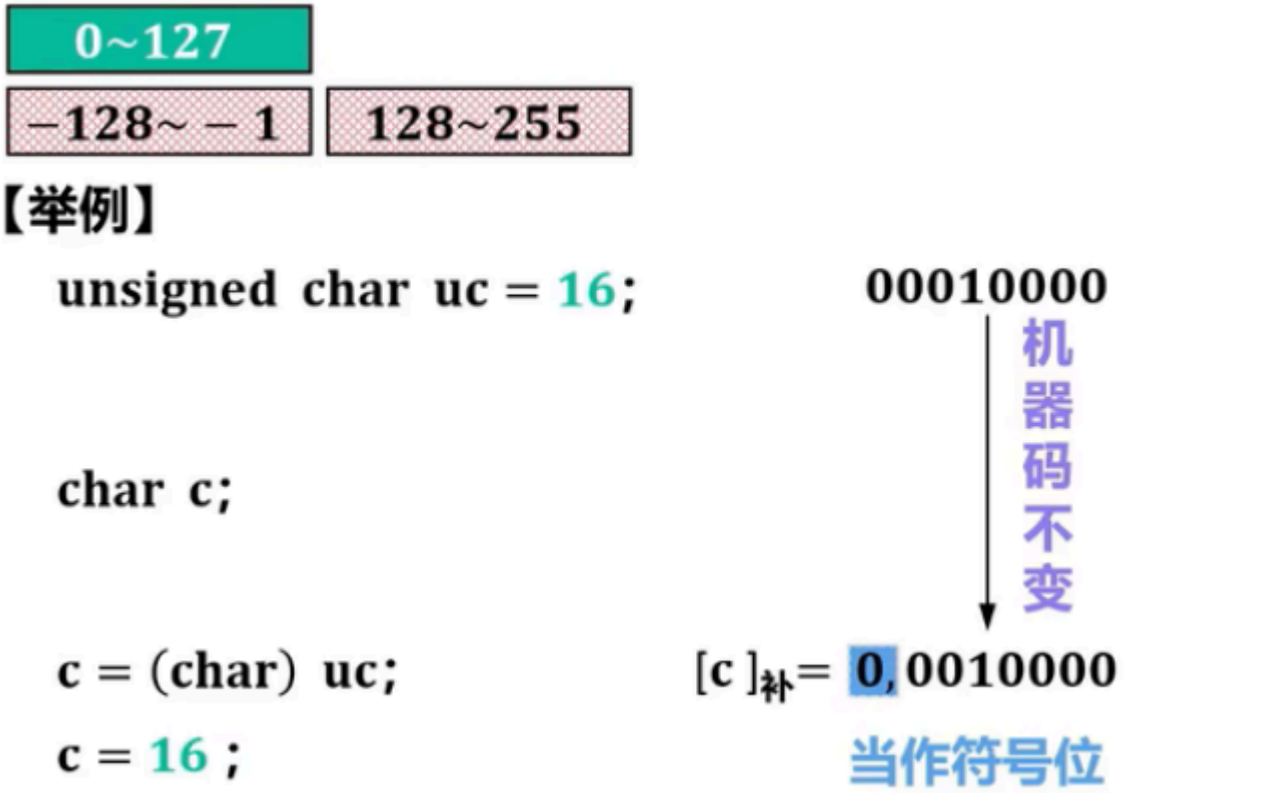


分三种情况

1. 重叠区域的转换：`char` 转 `unsigned char`

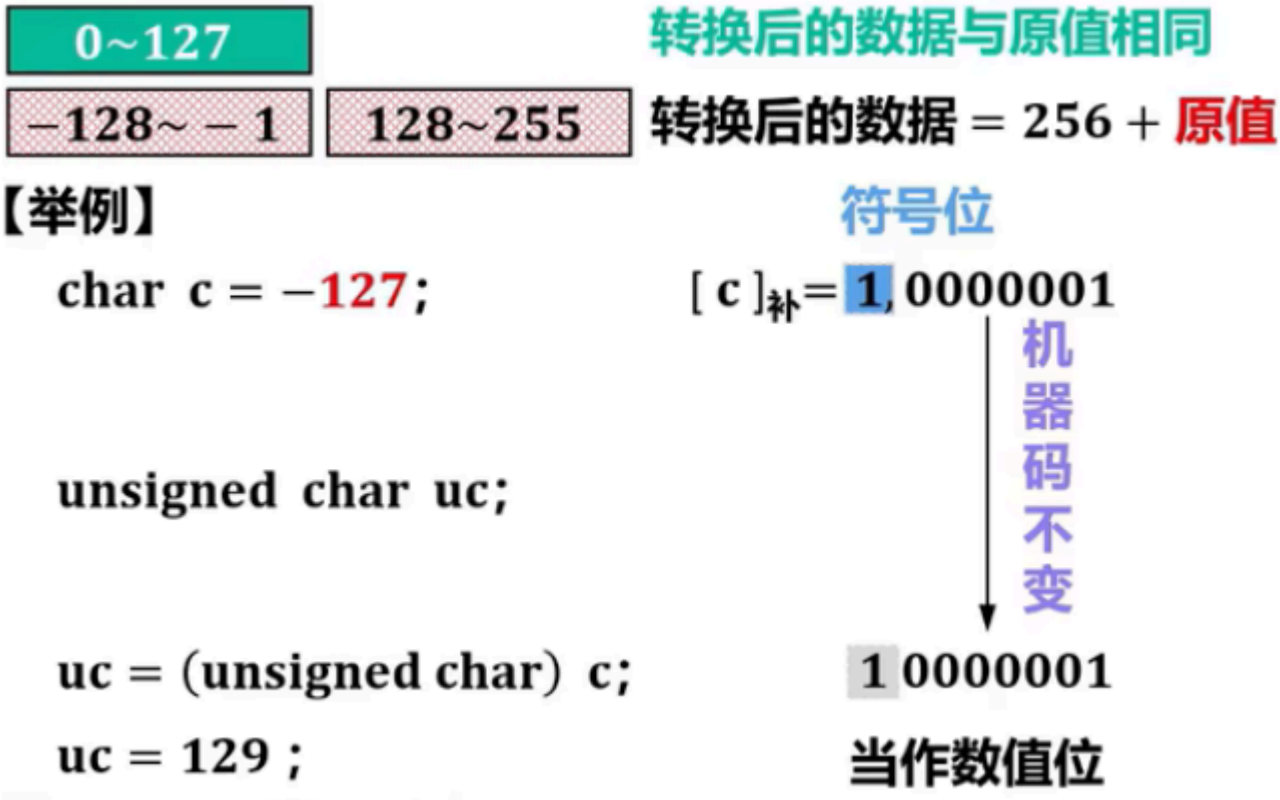


`unsigned char` 转 `char`



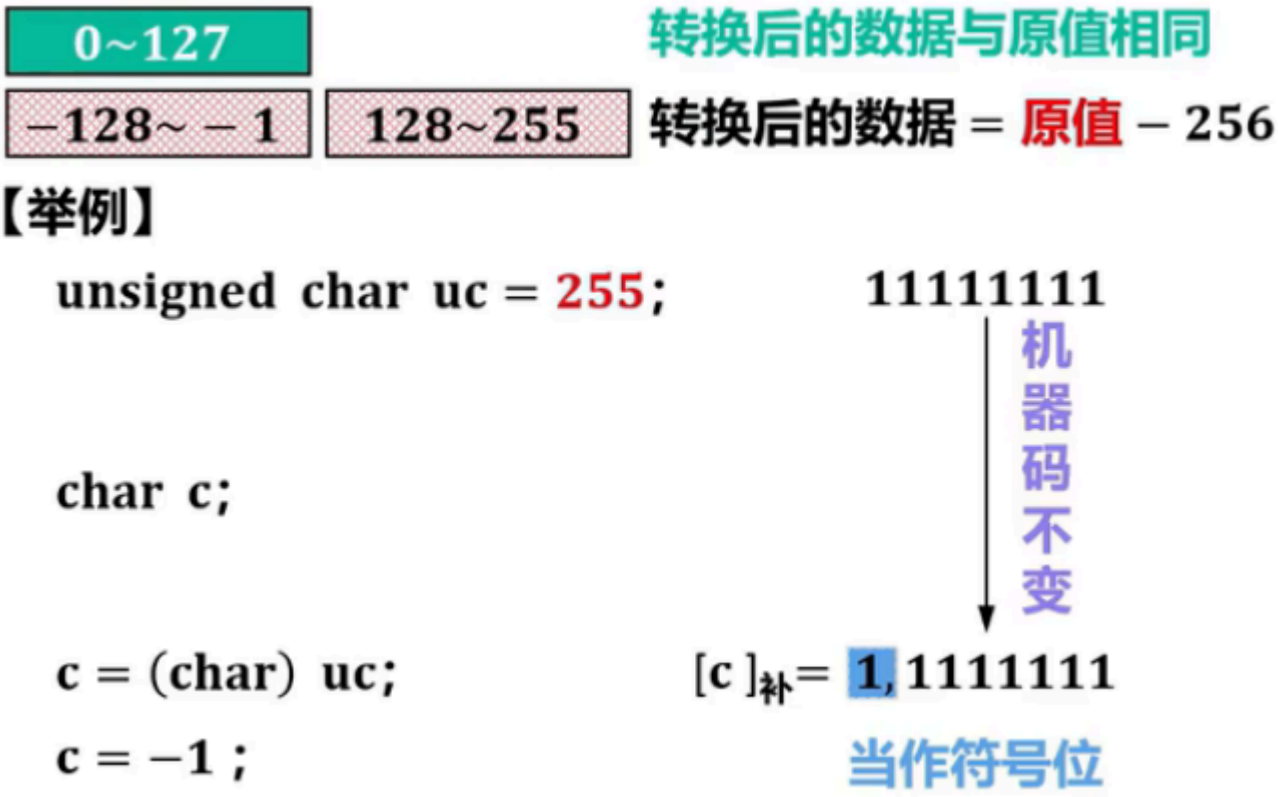
注：重叠区域为 0 到 127 时，最高位为 0；无论把最高位当符号位还是数值位，都不影响真值，所以重叠区域的转换不改变真值

2. 独属 `char` 区域： `char` 转 `unsigned char`



注：有符号转无符号，将最高位视为数值位；独属 `char` 区域的数均为负数，符号位为 1，视为数值位后贡献 256（8 位）

3. 独属 `unsigned char` 区域： `unsigned char` 转 `char`



注：无符号转有符号，将最高位视为符号位；独属 `unsigned char` 区域的数最高位均为 1，视为符号位后相当于减去 256

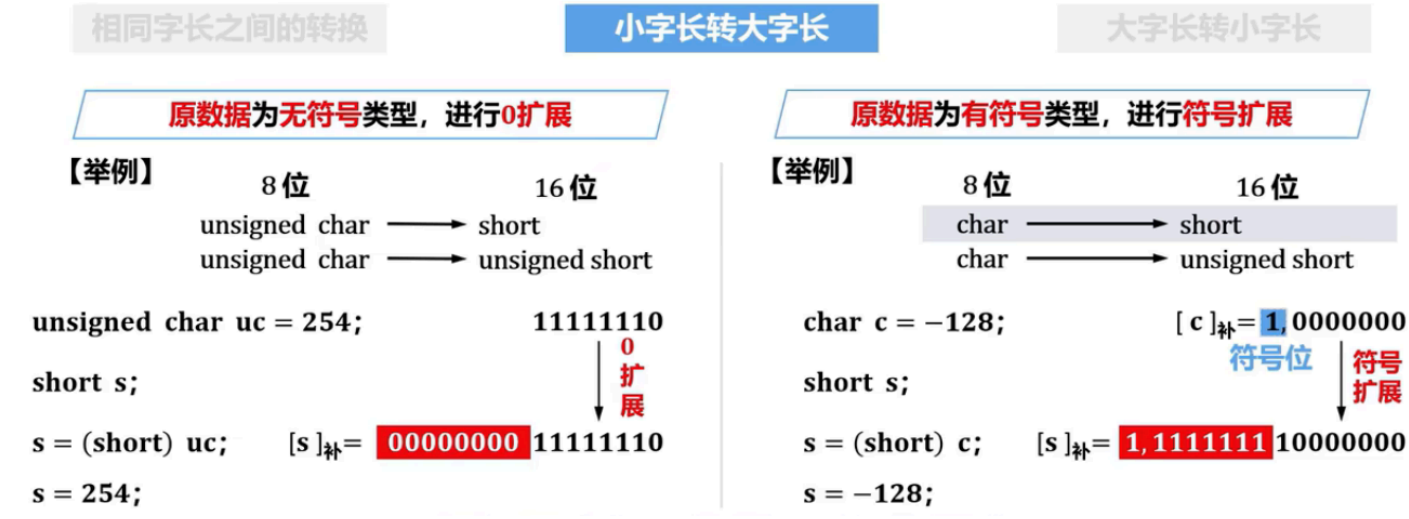
同字长转换公式

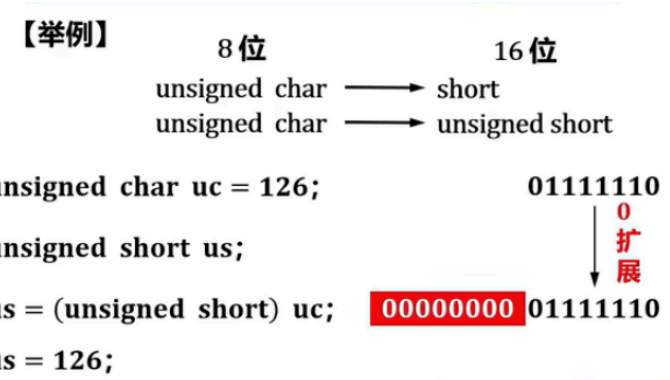
$$\text{位串}_{\text{目标}} = \text{位串}_{\text{源}} \bmod 2^N$$

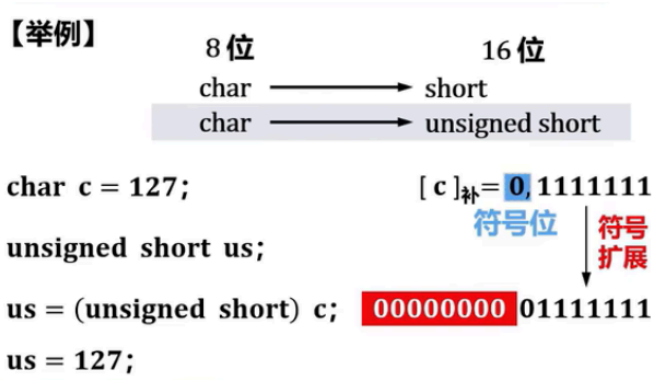
- 先按模 2^N 截取位串，再按目标类型解释
- 8 位时，-1 的位串 `1111 1111` 按无符号解释为 255，200 的位串按有符号解释为 -56

2.1.2 小字长转大字长

■ char、short、int、long 这4种整型数据的表示范围不一样，很可能数据转换后精度缺失，此时就只能尽量保持转换前后的机器码相同或机器码部分相同。







注：小字长转大字长的核心在于扩展

- 原数据为无符号：零扩展，高位补 0，**真值不变**
- 原数据为有符号：符号扩展，高位补符号位，**真值不变**

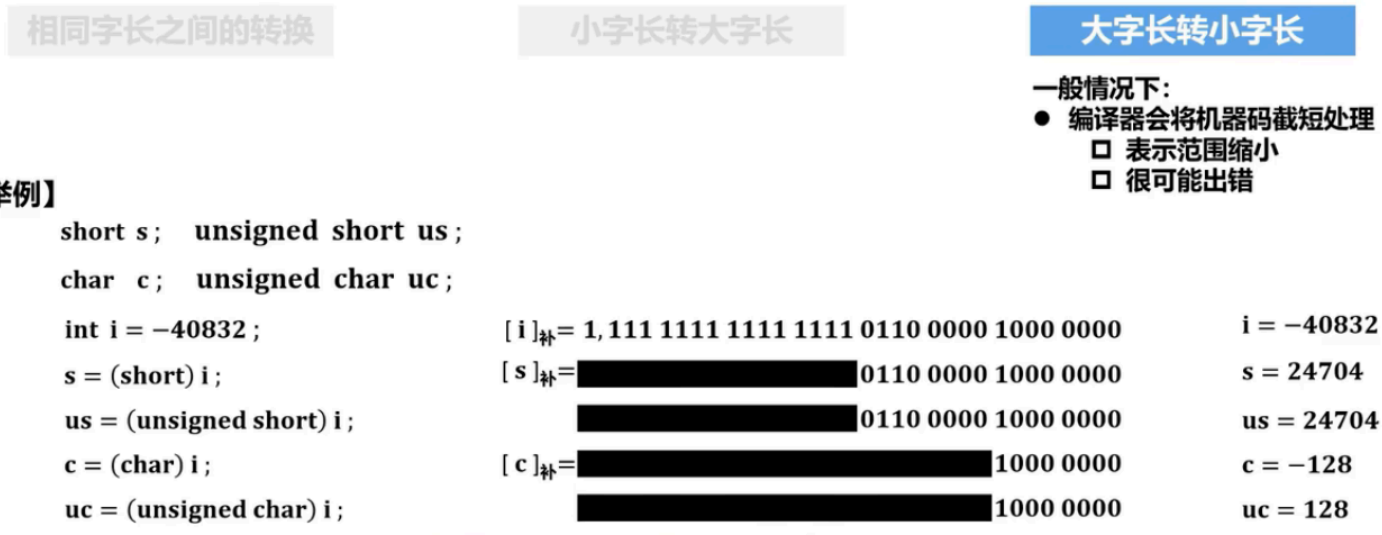
重要纠错

- 原笔记称“有符号负数符号扩展会改变真值”，这是错误的
- 符号扩展对正数和负数都保持真值不变；零扩展才会把负数解释成正数
- 例：`char -1`（`1111 1111`）→ `short -1`（`1111 1111 1111 1111`），真值仍是 -1
- 若把 `1111 1111` 零扩展成 `0000 0000 1111 1111`，真值变成 255，这才是改变真值

判断口诀

- 源类型无符号 → 零扩展
- 源类型有符号 → 符号扩展
- 目标类型不影响扩展方式，源类型才决定

2.1.3 大字长转小字长

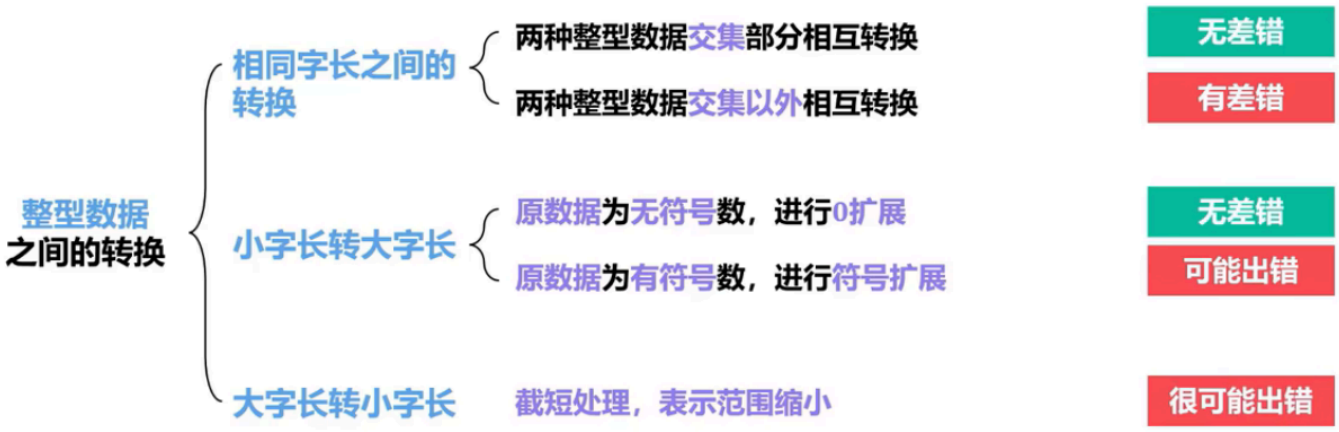


规则

- 直接截断高位，保留低 N 位
- 真值可能改变
- 有符号/无符号都按位截断，再按目标类型解释

例：32 位 `0x0000 01FF` 转 `char`，保留低 8 位 `0xFF`，按有符号 char 解释为 -1

2.1.4 整型之间转换的总结



转换	扩展/截断	真值
同字长有符号 ↔ 无符号	按位重新解释	可能改变
短 → 长，源无符号	零扩展	不变
短 → 长，源有符号	符号扩展	不变
长 → 短	截断低位	可能改变

易错

- 扩展方式由“源类型”决定，不是目标类型
- 同字长转换不改变位串，只改变解释
- 长转短不是“四舍五入”，是截断
- 有符号溢出在 C 中是未定义行为；无符号是模回绕

2.2 int、double、float之间的转换

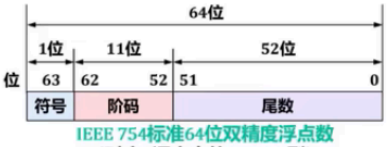
- int、float、double之间也可以进行强制类型转换。
  - 上述3种类型数据的机器码并不相同（int型数据是32位有符号整数，用补码表示；float和double型数据分别是32位和64位浮点数，它们的阶码用移码表示、尾数用原码表示）。
  - 上述3种类型数据的表示范围和精度也不相同。
  - 因此在转换过程中编译器只能保证数值尽量相等，大多数情况下只是近似值。
  - 下面，我们讨论以下几种转换情况。
    - float → double
    - double → float
    - float/double → int
    - int → float
    - int → double

2.2.1 float转double

float → double　　double → float　　float/double → int　　int → float　　int → double



IEEE 754标准32位单精度浮点数
(对应C语言中的float型)



IEEE 754标准64位双精度浮点数
(对应C语言中的double型)

- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示，偏置常数单精度为127，双精度为1023。
- 尾数：定点小数，用原码表示。符号位前移到最左侧。相邻左侧隐藏一个1，表示数值而不表示符号。对于单精度，尾数实际有24位；对于双精度，尾数实际有53位，但不保存隐藏的那个1，节省的比特位可用于提高尾数的精度。

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

由于double型数据的阶码和尾数的位数都比float型大，因此其表示范围更大、精度更高，转换后的double型数据与原float型数据的值完全相等。

- float 的尾数 23 位，double 尾数 52 位
- float → double 通常精确，不会丢失已有精度
- 阶码范围扩大，不会溢出

2.2.2 double转float

float → double　　double → float　　float/double → int　　int → float　　int → double

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

- 大数转换：可能发生溢出。例如：`double d = 1234567890123456; float f = (float) d;`
- 高精度数转换：发生舍入。例如：`double d = 1.123456789; float f = (float) d;`

- 尾数从 52 位截到 23 位，可能丢失精度
- 阶码范围变小，可能上溢/下溢
- 需要舍入

2.2.3 float/double 转 int

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times(2-2^{-52})\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

→

C语言中的int（32位）		
无符号(unsigned)最大值	机器码	0xFFFF FFFF
	真值	4,294,967,295
有符号最大值	机器码	0x7FFF FFFF
	真值	2,147,483,647
有符号最小值	机器码	0x8000 0000
	真值	−2,147,483,648
−1	机器码	0xFFFF FFFF
0	机器码	0x0000 0000

- 小数部分：向0方向截断
- 大数转换：可能发生溢出

- 向 0 截断，不是四舍五入
- 超出 int 范围时，C 标准是未定义行为
- 负数截断向 0，例如 -3.7 → -3

2.2.4 int 转float

float → double　double → float　float/double → int　int → float　int → double

C语言中的int（32位）		
无符号(unsigned)最大值	机器码	0xFFFF FFFF
	真值	4,294,967,295
有符号最大值	机器码	0x7FFF FFFF
	真值	2,147,483,647
有符号最小值	机器码	0x8000 0000
	真值	−2,147,483,648
−1	机器码	0xFFFF FFFF
0	机器码	0x0000 0000

→

数值的分类			单精度（32位，对应C语言中的float）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times(2-2^{-23})\approx 3.4\times 10^{38}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))\approx -3.4\times 10^{38}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$

- 两种类型都是32位，各自的数据组合（状态）数量相同，但二者在数轴上表示的数据并不完全重叠。
  - 由于float型浮点数的尾数包括隐藏位在内共24位，当int型数据的高8位（24~31位）数据为非0时，无法精确转换成24位浮点数的尾数，此时发生精度溢出。

- float 有效精度 24 位
- 32 位 int 超过 2^{24} 时可能丢失低位
- 可能发生舍入

2.2.5 int转double

float → double　double → float　float/double → int　int → float　**int → double**

C语言中的int（32位）		
无符号(unsigned)最大值	机器码	0xFFFF FFFF
	真值	4,294,967,295
有符号最大值	机器码	0x7FFF FFFF
	真值	2,147,483,647
有符号最小值	机器码	0x8000 0000
	真值	−2,147,483,648
−1	机器码	0xFFFF FFFF
0	机器码	0x0000 0000

→

数值的分类			双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1 = 2^{1023}\times(2-2^{-52}) \approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-1023}\times 1.0 = 2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-1023}\times 1.0 = -2^{-1022}$
		最小值	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1 = -(2^{1023}\times(2-2^{-52})) \approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11 = 2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01 = 2^{-1022}\times 2^{-52} = 2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01 = 2^{-1022}\times 2^{-52} = -2^{-1074}$
		最小值	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11 = -2^{-1022}\times(1-2^{-52})$

double型数据的尾数包含隐藏位在内有53位，可以精确表示所有32位整数。

- double 有效精度 53 位
- 32 位 int 可以精确表示
- 64 位 int 超过 2^{53} 时可能丢失精度

2.2.6 总结

C语言中数据类型转换 —— int、float、double之间的转换

float → double	double → float	float/double → int	int → float	int → double
值相同	大数可能溢出，高精度发生舍入	小数部分向0截断，大数可能溢出	比较大的数无法精确表示	值相同

【实例】假设变量i、f、d的类型分别是int、float和double，请判断下列C语言关系表达式是否恒为真?

`i == (int)(float) i;`	不恒为真	i 转换为float型时就可能出现精度缺失，可不考虑再转回int型
`f == (float)(int) f;`	不恒为真	f 转换为int型时就就可能出现小数部分向0截断，可不考虑再转回float型
`i == (int)(double) i;`	恒为真	i 转换为double型后数值不变，因此再转回int型后数值不变
`f == (float)(double) f;`	恒为真	f 转换为double型后数值不变，因此再转回float型后数值不变
`d == (float) d;`	不恒为真	d 转换为float型时可能出现大数溢出和高精度舍入
`f == -(- f);`	恒为真	浮点数取负仅将其数符（最高位）取反即可。

转换	是否精确	风险
float → double	通常精确	无
double → float	可能丢失精度	上溢/下溢/舍入
float/double → int	截断	超范围 UB

转换	是否精确	风险
int → float	超过 2^{24} 可能丢失	舍入
int → double	32 位精确	64 位可能丢失

易错

- 浮点转整数是截断，不是四舍五入
- 整数转浮点可能丢失精度，不是“整数一定精确”
- double → float 可能溢出成 ∞
- C 语言不检测这些转换的范围，结果由实现/标准决定

知识关联：整型编码（2.4 定点数的补码）；IEEE 754（2.10 浮点数表示的IEEE 754标准）；C 语言溢出（2.11A C 语言不检测数据溢出的特性）；数据溢出（3.5 定点数的加减运算——溢出检测）